Is this tweet ironic?

Final project in natural language processing: a text classification challenge

Deliverables: notebook + `submission.csv`
Metric: macro-F1 on a hidden test set
Deadline and hand-in channel: announced in class

THE CHALLENGE

People often write the opposite of what they mean: "Great, another Monday." You will build a model that reads a tweet and decides whether it is **ironic (1)** or **not ironic (0)**. The words of an ironic tweet are often positive while the meaning is not, so a model that only counts words sees the surface. Your job is to find the features, the models and the evidence that see the gap.

THE DATA

Tweets from the SemEval-2018 irony detection task (Van Hee, Lefever and Hoste, 2018), annotated by linguists and republished in the TweetEval benchmark and on Kaggle. The course pooled the corpus, removed duplicates and split it once with a fixed seed. User names appear as `@user`, links as `http`; hashtags, emoji, capitals and punctuation were kept. About 48% of the tweets are ironic, in both files.

File	Tweets	Columns	Use
`data/train.csv`	3,450	id, text, label	explore, train, validate
`data/test.csv`	1,151	id, text	predict; labels are hidden

WHAT YOU HAND IN

1. **The notebook** `irony-final-project.ipynb`, completed and run from top to bottom (Restart & Run All), with your exploration, experiments, error analysis and report in the marked sections.
2. **Your predictions** `submission.csv`: one row per test id, columns `id,label`, labels 0 or 1, written by section 9 of the notebook.

Name both files with your name or student number. The notebook must reproduce your submission: the model your report names is the one that wrote the file.

RULES

- Every modelling decision is made on data split off from `train.csv` (a validation set or cross-validation). The test labels are never used, looked up or reconstructed; doing so scores zero for the project.
- Any Python library is allowed. Pretrained language models are allowed and encouraged if you declare them, except models already trained on this dataset (for example `cardiffnlp/twitter-roberta-base-irony`), which have seen the test tweets.
- AI assistants may help you; say how you used them, and be ready to explain every line of your notebook in person.
- Work individually unless told otherwise. Cite the dataset paper and every source you build on.

GETTING STARTED

Open the notebook in Google Colab with the badge in the course README (it downloads the data itself), or run it locally from the module folder with the course requirements installed (`pip install -r requirements.txt`). Sections 1 to 6 run as given: they load the data, set the evaluation rules and build the baseline that works. Your work is the exploration (3), the experiments (7), the error analysis (8), the submission (9) and the report (10). The checklist at the end of the notebook is the hand-in checklist.

HOW POINTS ARE AWARDED (100)

Part	What earns the points	Points
Exploration	At least two analyses of your own beyond the given ones, each with a short interpretation that later informs a modelling choice.	15
Method and validation	A clean validation protocol; at least three experiments beyond the baselines, one change at a time, all logged in the results table; cross-validation where runs are close; fixed seed; the notebook runs end to end.	25
Performance	Macro-F1 of `submission.csv` on the hidden test set. The given baseline scores about 0.64; points scale from there, with full marks for a clearly stronger model. Format errors cost points; a file the grader cannot read cannot be scored.	20
Error analysis	At least ten false alarms and ten misses read; the kinds of mistake named and counted; one fix tried and its effect reported honestly.	20
Report and code	Section 10 written in plain language a non-reader of your code understands: approach, results, what the model learned, limitations and ethics, AI-use statement, references. Readable code, no dead cells.	20

> **What a strong project looks like.** It beats the baseline by a margin larger than its own cross-validation spread, explains *why* the winning change helped with evidence from the data, admits which differences were noise, and shows ten errors the author actually read. A high score with no explanation earns the performance points and little else.

ADVICE

- Read tweets before computing anything; the six-tweet quiz in section 2 is there for a reason.
- Standard text cleaning deletes evidence of irony (capitals, punctuation, emoji, hashtags). Test it instead of assuming it.
- One validation split is one measurement. Two models two points apart are tied until cross-validation says otherwise.
- Macro-F1 moves when *both* classes improve; watch precision and recall of the ironic class, not accuracy alone.
- Start simple, log everything, and spend your last hour on the errors and the report, not on one more model.

Data: Van Hee, Lefever and Hoste (2018), SemEval-2018 Task 3, via TweetEval (Barbieri et al., 2020), github.com/cardiffnlp/tweeteval; Kaggle mirror: kaggle.com/datasets/thedevastator/tweeteval-a-multi-task-classification-benchmark. Course repository: github.com/maglionejm/fontys-tech-exercises, folder "M4 - Natural Language Processing" (README.md, data/, irony-final-project.ipynb, this guide).